

Change Report

Cohort 3 Team 3

Fares Almutairi
Taro Chandiwana
Lily Gaunt
Jacob Last
Alex Ponomarenko
Giulia Sclano

Part A

Planning and Process:

Before development started our team reviewed the documents inherited from the previous group to better understand the scope design and assumptions. First, the requirements documents were reviewed and updated and aligned with demographic and customer requirements. after which architectural changes were planned in parallel with implementation. The tasks were assigned equally across the team. with some members focusing more on documentation at different stages to ensure progress across all deliverables. At first we split the implementation and testing into two sub-groups, implementation team and testing team.

The architecture was started before implementation and evolved alongside it. This is to have better work flow as the map and screen layouts were changed. In addition, gameplay mechanics directly influenced later features such as events, the leaderboard, and achievements. Playing remained evolving throughout the project development with minor adjustments for necessary refactoring such as UI corrections and map layers.

Tools and Conventions

GitHub was used as the primary collaboration and version control platform for both code and documentation. commit messages were used to incrementally commit changes, making it easy to track updates made to inherited deliverables and requirements . While written deliverables were kept in PDF format for consistency.

A continuous integration (CI) approach was used to support collaborative development. We regularly update and synchronize code to the main branch. GitHub Actions such as push and pull requests were used to commit and automatically build and test the project to the main branch. This is to be sure that test failures are detected early before they cause problems and merge conflicts when multiple team members are working on the code at different times.

As for updating the assessment 1 deliverables, in order to keep track of changes we enforced a rule that whenever you changed something in an assessment 1 deliverable, you make a note of what you changed (and why you changed it) within this document, so that it can be properly written up at a later date. We also used a method where we highlighted all of the original deliverable content in a faint colour, and anything we added we made sure was not highlighted. This allowed us to clearly see anything that had been added and where it had been added without having to search through the Google Docs history. These two methods together eliminate any confusion and allow for a clear write up of changes and additions to the documents.

Change Tracking and Review

Updated documents clearly reference modified sections, and newly added content, with precise URLs provided for both the original and updated versions.

Changes to Requirements

Original: https://eng1-group3.github.io/website-2/public/original_deliverables/Req1.pdf

Updated: https://eng1-group3.github.io/website-2/public/updated_deliverables/Req2.pdf

Overview :

The updated requirements reduced the game scope by replacing map generation with a single pre-made map. In addition to zoomed-in camera perspective and many in-game interactions and incentives. Such as introducing competitive elements like a leaderboard system, score tracking, and in-game achievements.

Map :

UR_MAP & FR_MAP_LOADING: the scope of the game was reduced. The original requirements stated in **UR_MAP** that "There are several maps for the user to choose from and playthrough" ; this was changed to the game only loading one pre-generated map. Correspondingly, **FR_MAP_CREATION** was changed to **FR_MAP_LOADING** the game will "load a pre-generated map" rather than generating one. This change was made due to strict project deadlines, as procedural map generation would require significant additional implementation time and would not be feasible within the project constraints.

Scoring and competition:

UR_SCORING: the objective of the scoring system was changed for best social competition. The original requirement from **UR_SCORING** "can win by achieving the best score" was changed to **UR_SCORING** "climb up the leaderboard by achieving the best score" this change was done to clear ambiguity of the score being a condition of winning when stating "can win by". In addition to having a leaderboard linked to the score.

UR_ENDING: the requirement was expanded to show the player score and the leaderboard.

UR_LEADERBOARD & FR_LEADERBOARD_DISPLAY: these are new requirements added. The game now must store, display and sort the **top 5** scores from highest to lowest.

FR_LEADERBOARD_NAME_INPUT: a new functional requirement was added requiring the game to ask the user for a username.

Gameplay Mechanics :

UR_DIFFICULTY & FR_EASY_DIFFICULTY: These requirements were removed to reduce scope and implementation complexity, similar to the reasons behind the **UR_MAP** changes.

UR_PAUSE: The restrictions on pausing were removed. The original **UR_PAUSE** was "Game should be able to be paused at least on low difficulty" This was changed to "Game should be able to be paused" this is due to the removal of **UR_DIFFICULTY** and **FR_EASY_DIFFICULTY**.

UR_CAMERA: the camera perspective was changed to have better immersion for the player and add some difficulty to compensate for the **UR_DIFFICULTY** removal. The update specifies the player should see the map "zoomed in".

UR_RESTART & FR_RESTART: a requirement was added to improve game flow . The update notes the addition of a "way to restart the game". The game can restart when the it halts eg game win screen, the loss screen and pause screen

UR_EVENT_COUNTER: added a requirement to have a counter displaying the number of times the user has encountered each type of event in every game cycle.

FR_NEGATIVE_EVENTS: the number of negative events was increased from 2 to 5 to align with client expectations and enhance player engagement.

UR_ACHIEVEMENTS & FR_ACHIEVEMENTS_DISPLAY: a new requirement was added to enhance the player experience and progression. The system must reward the player with achievements when hitting specific goals and.

NFR_GAME_TIME: A logical error in the requirements was fixed. The original Non-Functional Requirement stated a "minimum of 5 minutes" (≥ 5). which was contradicting the **UR_TIME** of a "5 minutes maximum" requirement.

FR_MAIN_MENU: The menu terminology was updated for consistency, changing the required button label from "New Game" to "Play"

UR_SYSTEM_REQUIREMENTS was adjusted to be more specific on windows and linux systems. This is because having the game run on any OS is unrealistic and does not align with the client specification.

NFR_SYSTEM_RESTRICTIONS fit criteria "runs on 99% of machines" was changed to "runs on most modern machines" + "no memory leaks" because the original fit criteria is very vague and hard to measure.

Changes to Architecture

Original: https://eng1-group3.github.io/website-2/public/original_deliverables/Arch1.pdf

Updated: https://eng1-group3.github.io/website-2/public/updated_deliverables/Arch2.pdf

All page numbers below treat the title page as page 0.

Overview

The first thing we did when updating this document was to add headings in order to break sections up into clearer sub-sections. We also increased the size of all images in order to make them a readable size.

We also added an 'Overview' on page 1 to describe what our goals for the updated architecture are, including design principles we're aiming to follow so we can stick to this. It also discusses how we'll be aiming to abstract the architecture to a high-level scope, rather than focussing on implementation details.

Changes/Additions to the Structural Diagrams

Changes to Class Diagram, page 2 and 3

Within the project, the original structure of the architecture was designed well, allowing us to easily create more classes. An example of this is the 'Event' abstract class, which we could extend to make new events (abstracted out of the class diagram). The original class diagram, however, did not capture this structure, and was not very detailed. Because of this, we decided to add more classes to the diagram, to offer a high-level view of the system, whilst including enough details to allow a good understanding of the structure of the system. The classes added to the diagram include Achievement, Leaderboard, LeaderboardEntry, SortLeaderboard, InputHelper and EventCounter. Another change we made was specifying the relationships between the classes, as this allowed the logic of the system to be easier to follow through.

Addition of Class Stereotypes, page 2

Within the class diagram, we added class stereotypes for each of the classes, with all classes fitting into three stereotypes - Controller, Boundary and Entity. We felt that this was necessary to add to the range of information given by the diagram. We also found it beneficial when adding more code, as they helped us understand the role of each class within the program.

Addition of Design Patterns, page 4

We identified the design patterns of some of the classes within the system as a means of improving reusability and maintainability. We chose to do this for the most important classes of the system to keep it efficient, as those classes are most connected to other classes and have the most roles and thus fit the design patterns the strongest.

Changes / Additions to Behavioural Diagrams

In the original version of the architecture document, the only behavioural diagram that existed was an overly-detailed and difficult to interpret UML state diagram. This diagram doesn't succeed at communicating information such as requirements the system meets. Therefore, we updated this to make it clearer, along with adding a few more behavioural diagrams so that more of the system is covered than just the 'Playing' state of the game.

Addition of a Use Case Diagram, page 5

Firstly, we created a use case diagram in order to communicate all of the actions that the user should be able to perform within our game.

This was useful as it's a simple diagram which helps to verify the requirements our system will meet (which have been listed below the diagram).

Also, because it's a non-technical diagram, it can be shown to a stakeholder in order to give a high-level overview of what our system will be able to do. This allows them to point out if we're missing anything that they require the game to be able to do before we finish implementation.

It also gave us ideas about what parts of the game would be useful to create further behavioural diagrams about.

Addition of an 'Overall Game' State Diagram, Page 6 and 7

Before trying to improve the existing state diagram, which focussed on states during gameplay, we first saw it necessary to produce a state diagram which covers all of the states the system can be in from a higher level. Therefore we produced a simple state diagram to include all of the screens the game can be displaying, and how the user should be able to transition between them.

This helped to verify more requirements (listed below the diagram) and ensured that all screens of the system can actually be reached by the user in an easy and logical way.

Changes to the 'Playing' State Diagram, Page 7 and 8

As already discussed, the previous version of this diagram went into far too much implementation detail, referring to specific attributes and methods in the code. This goes against our goal for this architecture document being to a fairly high-level of abstraction. Therefore, to start updating this diagram we first replaced all state transitions with simple phrases that describe what causes the state transition.

We also moved the 'Paused' state inside of the 'Playing' state, as it is a state the game can go into whilst playing, not separate. We created an 'Unpaused' state which contained the other substates.

The previous state diagram also had state transitions from inside of substates to the inside of other substates, making it extremely difficult to follow. Therefore, we fixed this by making sure transitions were kept between substates in the same state-space. This makes the diagram a lot easier to understand and reduces the visual clutter of the diagram as arrows aren't crossing across the whole diagram.

We removed the 'Item' and 'Colliding' substates within the 'Player' substate, as it doesn't make sense the player can be in an 'Item' state, and it doesn't make logical sense for the player to go into a 'Colliding' state - instead collisions should just be handled within player movement.

Finally, we reworded the discussion so that it supported the updated diagram, keeping the relevant information about how this state diagram supports the event-driven architecture and highlights how different parts of the system work together. We also added a more explicit link back to the requirements this diagram helps to support.

Addition of a 'Player Movement' Sequence Diagram, Page 8 and 9

As discussed above, it doesn't make sense for the player to go into a 'Colliding' state - instead it makes more sense for this to be handled when the player is moving.

Therefore, we produced a sequence diagram for player movement, helping to outline how collisions could be handled within the architecture of the game. This includes what actors need to interact and the information that should be passed between them in order to handle the collision appropriately.

As before, we also linked this diagram back to the requirements it helps to support.

Changes to Method selection and planning

Original: https://eng1-group3.github.io/website-2/public/original_deliverables/Plan1.pdf

Updated: https://eng1-group3.github.io/website-2/public/updated_deliverables/Plan2.pdf

The planning deliverable was updated to show our planning process and evolution in the second assessment of this module. The updates included:

1. Edited the [CONTRIBUTING.md](#) file (link and file itself)
 - Section modified: Within 'Code-specific collaboration' heading
 - Changes: changed the link to point to our own version of the file in our repository, as well as within the file itself, removing the naming convention for branches that the previous team defined, as we did not end up using this, even though it may have been a good idea as it would have made our project more professional.
2. Removed CI section
 - Justification: irrelevant to us and have whole CI document
3. Added planning methods
 - Section added: Everything under the 'Methods' heading
 - Changes: Added this section as the marking for Assessment 1 states 'Give an outline and justification of the team's software engineering methods' and the document only included outline and justification of tools used. This did not change from assessment 1 as we still used the agile (scrum) methodology.
4. Added Assessment 2 Systematic plan
 - Section added: Everything under the 'Systematic Plan (Assessment 2)' heading
 - Changes: Added the final version of the gantt chart we developed throughout the second half of the assessment, as well as how this plan evolved.

Regarding everything else that was unchanged; the outline and justification of the tools the previous team used was not changed after reading through it as they used the same tools as us and we decided that their justification was suitable for us as well. The content under the 'Systematic Plan (Assessment 1)' was also not changed as their timeline and process of their part of the assessment was obviously not altered.

URLS:

Changes to Risk Assessment and Mitigation

Original: https://eng1-group3.github.io/website-2/public/original_deliverables/Risk1.pdf

Updated: https://eng1-group3.github.io/website-2/public/updated_deliverables/Risk2.pdf

The deliverable for risk assessment has been updated based on the revised and expanded requirements included in the second iteration of the game project. Five new risks, ranging from R011 to R015, have been included to mitigate requirements related to new features, with an existing risk, R005, being further specified to mention new features.

Justification and Specific Updates:

1. Updated Existing Risk (R005 - Page 4 of the updated document)

Section Altered: Risk R005, "Feature doesn't fit what the customer envisaged"

Changes:

The Consequences section has been amended to make explicit the assertion that: "With new functionality like leaderboards and achievements, alignment is essential."

The Mitigation strategy has been revised to add: "Use prototypes or mock-ups for new functionality like achievements and leaderboards."

Justification: There are now complicated features (leaderboards and achievements) being added, requiring alignment with customer needs. The original approach to mitigation was general; the revised one outlines specific methods for handling these highly visible features.

2. New Risks Introduced (R011-R015 - Page 5 of the revised document)

R011 – Leaderboard data persistence problems (Medium/Medium)

Rationale: The presence of UR_LEADERBOARD requires a permanent storage mechanism for scores within a game instance or between instances, since the loss of scores would affect the competitive nature of the game and would have a moderate level of likelihood, which would occur as a result of file input and output during the development of a game.

R012 – Achievement system bugs

Medium/Low

Justification: The UR_ACHIEVEMENTS requirement adds unlock conditions that need to be triggered. Unlock achievements have a potential vulnerability to edge cases and/or logical flaws (Medium risk) that would not have as much impact as core gameplay-related flaws.

R013 - The 'Restart game' function is not working properly. Justification: UR_ENDING has been updated to require "a way to restart the game." Handling the state for such restart functionality is complex, although its implementation is easy (Low likelihood). Its failure would force the user to manually exit and restart the application (Medium impact).

R014 – Leaderboard Display Issues (Medium/Low) Rationale: FR_LEADERBOARD_DISPLAY

requires proper sorting, formatting, and display of scores. Bug presence in the user interface and sorting irregularities are likely found in leaderboard implementations (Medium likelihood), although display problems have lower impact than data loss (Low impact).

R015

– Single map becomes repetitive (High/Medium) Rationale: The first changed requirement, moving from “several maps” to “the game has one map” as part of UR_MAP, has a high probability of repetitiveness that directly goes against the presence of the leaderboard feature to encourage repeated play. The effect is moderate because there can be other ways to offer game diversity.